At agent volume, green is a claim. It becomes evidence only when a check the agent cannot touch agrees.

Friday, October 16, 2026

Agent volume reached source control this week, and the research turned on the tests themselves: generated tests that lock bugs in, agent histories that leave suites red, and benchmarks that score agent tests twice as kindly as they should.

THE STORY OF THE WEEK

Anthropic replaces git because agents merge 30,000 changes a day

Last week GitHub rebuilt its storage for agent traffic. This week Anthropic went further and moved its monorepo off git entirely. Daily merges grew from about 2,500 in May 2026 to more than 30,000 in September, and CI alone performs about 25 million checkouts a day. Anthropic estimates naive git fetching would mean more than 600 PB of traffic a day.

A team of seven built Scion, a server-centric version control system, and Antique, a new code review tool, and completed the migration in five months. Checkouts went from minutes to seconds. Those performance figures are self-reported, with charts but no methodology, and Scion is internal only.

The quality lesson sits in two quieter decisions. First, a global merge-queue lock could not keep up at agent scale, so the team stopped trying to prove every change safe before merge and invested in post-merge and pre-deploy automated testing instead, accepting that concurrent changes will sometimes conflict. Second, Antique encodes semantic approval rules that separate comment-only edits from sensitive logic changes, and human reviewers now judge the intent of a change rather than reading every line. Adversarial AI review fills the gap, which the team calls effective but sometimes myopic.

What we can learn: at this volume, the pre-merge gate stops being the place where quality is decided. The deploy gate and the tests behind it become the real control, and review becomes a risk-routing decision rather than a reading exercise.

> **Takeaway:** Ask clients which gate actually stops a bad change today. If merge volume triples, the answer quietly moves to whatever runs after merge.

THREE READS WORTH YOUR WEEKEND

Are Harnesses Worth the Cost?

Naresh Jain, martinfowler.com, Oct 13

A rare controlled experiment on what an agent harness buys. Ten paired runs per model built the same Java backend-for-frontend, and an outside evaluator judged every run against 178 acceptance tests the harness never saw.

Most interesting: For the frontier model the harness changed nothing (10/10 either way) while multiplying tokens tenfold. For the cheaper model it lifted acceptance from 3/10 to 10/10, and the harnessed cheap model still cost about 33% less per run than the unharnessed frontier one. Small sample, one greenfield service, costs estimated.

Why it matters: Harness design is a cost decision tied to model choice, and the hidden acceptance suite is the part that never changes.

A green exit code is not evidence that the work happened

Chase W. Hughes, Stack Overflow Blog, Oct 8

Tools earned our trust by failing loudly. Unattended agents can exit cleanly and report success without producing anything, so an exit code becomes the agent's self-assessment, not an observation of the work.

Most interesting: The recommendations are directly testable: assert on state the agent did not write (the database row, the file, the pushed branch), judge the artifact rather than the transcript, alert on "successful" scheduled runs that changed nothing, and measure pass^k before deployment.

Why it matters: This is the checklist for any agent job a client runs on a schedule.

nobody should type code by hand anymore. here's what's left.

Geoffrey Huntley, Oct 13

A long, opinionated essay arguing that with models no longer the bottleneck, what remains is vision, verification, and agency. Many claims are his own and unmeasured.

Most interesting: The concrete part, which Simon Willison singled out: AGENTS.md instructions drift out of sync with the code, so Huntley replaced one with a pre-commit hook that scans the diff for code bypassing his diagnostics library and rejects the commit with instructions. Compliance went to ten out of ten.

Why it matters: A rule written as prose is a request. A rule written as a hook is a test.

FROM THE LAB

Do Modifications Made by Coding Agents Make Subsequent Software Maintenance Harder?

Pan, Zhang, Wu, Zhang, Zhang, Zhang | preprint, Oct 10

Across 100 maintenance episodes in 50 Python repositories and 1,996 paired follow-up tasks, starting from agent-written history instead of developer history lowered later success by 5.6 points, and by 14.0 points when earlier tests also had to pass. Two-thirds of agent-history starting points already failed earlier tests. When

the inherited code passed its earlier tests, the gap disappeared.

For practice: The maintenance cost of agent code is mostly the broken tests it leaves behind. Make "every prior test still green" an exit gate for agent changes, not a hope.

SWEeper-Bench: Can Agents Discover Bugs in Interactive Software?

Yao, Chen, Kang, Narasimhan, Liu | preprint, Oct 2026

200 real bug-fix tasks across 200 web apps, with no bug report given. The best of 15 frontier agents solved 59.0%; given the bug description, agents passed about 96%. Each fix is checked in a browser with a target test and a preservation test.

For practice: Agents repair well and discover poorly. Exploratory testing, the part that finds the failure in the first place, is where human QA still adds the most.

One Skill Too Many: How Co-Installed Skills Conflict in Coding Agents

Yan, Xu, Li, Xu, Liu, Deng, Ma | preprint, Oct 8

Mining 20,947 repositories, about one in four installed skills sits next to a similar one. In 6,368 runs on three models, the similar skill took about one run in five from the intended one, those runs lost over a third of the intended skill's exclusive functions, and the final reply named the substitution in only 0.9% of cases.

For practice: Skill sprawl changes agent behavior while tasks still look complete. Inventory skills, log which one ran, and test for the functions only the intended skill provides.

FROM THE FIELD

Meta builds a standalone app with agents inside a verification loop

Meta's Forum team describes building the Groups companion app with agents running plan, implement, build, test, and correct, reaching core feature parity with Facebook Groups in eight weeks.

The verification design is the useful part. A linter runs after every agent turn, tests must pass before a pull request is created, and a task is complete only when automated tests and visual checks pass. Pull requests carry build results, test outcomes, and recorded verification sessions. Navigation agents use the live app like a person and report regressions, and repeatable stress scenarios found a memory issue whose fix roughly halved memory use and page faults.

The honest gaps: no team size, code volume, or defect data, and every shipped change still carries an engineer's sign-off. The agent also edited the wrong screen until it got tools to inspect the live view hierarchy, and its investigations were weak until it received the same code pointers and design docs a new engineer gets. Their rule is worth borrowing: when the agent repeats a mistake more than twice, encode the correction once, for everyone.

THE QA LENS

Generated regression tests can lock bugs in

Charoosaei, Richter, and Papadakis (preprint, Oct 8) generated regression tests for 145 pull requests in SciPy, Qiskit, and pandas. Between 8% and 17% of the tests were fault-enforcing, asserting the buggy behavior as correct, while only 2.4% to 4.8% revealed a fault. Most of the bad tests survived: 83% to 91% still passed several commits later, and developer-written suites caught only 14% to 30% of the faults they protected.

A second paper explains why this slips through. TestPrism (preprint, Oct 8) judges agent-written tests against 3,000 candidate implementations, half valid and half invalid. Across 14 coding-agent configurations, tests looked successful 59.67% of the time against a single reference solution, but only 28.00% both accepted every valid version and rejected every invalid one. And IBM Research's Tangent study (ASE 2026) of 2,572 test methods in open-source agent applications found mostly narrow, heavily mocked unit tests with shallow checks.

QA's opportunity is treating a generated assertion as a claim to review, not a fact to keep: run new assertions against mutants or alternate implementations, and require a human look before a generated test can pin current behavior on changed code.

ONE BIG IDEA

> ***At agent volume, green is a claim. It becomes evidence only when a check the agent cannot touch agrees.***

WEEKEND RABBIT HOLE

Bootstrap triages its backlog with agents that cannot act

What it is: Julien Deramond's first-hand account (Oct 15) of building twbs/triage with Claude Code and background agents. Agents analyze issues and pull requests, draft replies, score each item's ease from 0 to 10, and prepare up to three fix branches. The backlog fell from more than 500 items a few months ago to 230, and nine open issues turned out to be already fixed.

Why it was selected: it is a clean, small-scale example of policy separation. Agents never check out or run contributor code, reproductions are built from scratch against Bootstrap's own build, and the only automatic action is setting labels from a fixed allowlist. Every decision stays with maintainers.

Pay attention to: how much value comes from read-only analysis alone, and the gaps the author names: time saved was not measured, examples are selected successes, and agents lack the history that lives in maintainers' heads.

FUTURE BET

BET: By the end of 2027, at least one mainstream AI test generation tool will block generated regression assertions on changed code from being committed until they pass a mutation or alternate-implementation check, or a human approves them.

WHY: Up to 17% of generated regression tests enforce faults and most survive for many commits, and single-reference checks roughly double the apparent quality of agent-written tests.

SOURCE: Charoosaei et al. (arXiv 2610.11835), TestPrism (arXiv 2610.12289)

Bet check. *A widely cited coding-agent leaderboard will score submissions against held-out, requirement-derived tests:*
Strengthening. TestPrism shows single-reference evaluation overstates test quality, and Jain's harness experiment used 178 hidden acceptance tests as the only acceptance authority.

RADAR CHECKPOINT

Decision: add **Agent run provenance records** (Governance, Experiment now). In one week Claude Code, GitHub Agentic Workflows, and Codex each shipped part of it: model, prompt, and tool records per request, permission mode and verdict per tool decision, rejection of silent model swaps, and approvals restored on resume. Default-deny network for agent jobs stays a candidate, to be decided next week as its own item or as part of Policy separated CI agents. Agent-written orchestration programs and page-exposed agent tools (WebMCP) stay candidates: one vendor each so far.

View the current SDLC Radar